

Digital Image Analysis Tutorial 9

```
1  """AI531C T09 studio – The Shape & RF Calculator (Tue 6 Oct, 2-3 PM).
2  INPUT: none (pure arithmetic, verified against real convolutions).
3  OPERATION: build two functions – out_size() and rf() – and CHECK them against
4  | actual cv2/scipy convolutions; then size an encoder for a target object.
5  PARAMETERS: the L22 trace (224 input) and the L23 recurrence.
6  EXPECTED OUTPUT: printed traces (asserted against scipy ground truth).
7  INTERPRETATION: two tiny functions replace guesswork for every architecture you read.
8  """
9  import numpy as np
10 from scipy.signal import convolve2d
11
12 def out_size(n, k, s, p): return (n + 2*p - k)//s + 1
13 def rf_trace(layers):
14     rf, jump = 1, 1
15     for k, s in layers:
16         rf += (k - 1)*jump; jump *= s
17     return rf
18
19 # [1] verify out_size against a REAL convolution
20 x = np.zeros((224, 224))
21 y = convolve2d(x, np.ones((7, 7)), mode="valid")[:,2, ::2] # k7 s2 p0 (valid)
22 pred = out_size(224, 7, 2, 0)
23 print(f"[1] k7 s2 p0 on 224: scipy gives {y.shape[0]}, out_size() gives {pred}")
24 assert y.shape[0] == pred
25
26 # [2] the L22 trace, reproduced by YOUR function
27 trace = [(7,2,3),(3,2,1),(3,1,1),(3,2,1)]
28 n = 224
29 for k, s, p in trace:
30     n = out_size(n, k, s, p); print(f"    k{k} s{s} p{p} -> {n}")
31 assert n == 28
32
33 # [3] receptive fields
34 for layers, name in [( [(3,1)]*2, "two 3x3"), ( [(3,1)]*3, "three 3x3"),
35 | [(3,1),(3,2),(3,1),(3,2),(3,1)], "with strides"]]:
36     print(f"[3] RF of {name}: {rf_trace(layers)}")
37 assert rf_trace([(3,1)]*2) == 5 and rf_trace([(3,1)]*3) == 7
38
39 # [4] design round: how deep until RF >= 100 (plain 3x3)? and with stride-2 every 2nd?
40 d = 0; layers = []
41 while rf_trace(layers) < 100: layers.append((3,1)); d += 1
42 print(f"[4] plain 3x3 layers for RF>=100: {d}")
43 layers2, d2 = [], 0
44 while rf_trace(layers2) < 100:
45     layers2.append((3, 2 if d2 % 2 == 1 else 1)); d2 += 1
46 print(f"    alternating stride-2:    {d2} layers")
47 assert d2 < d
48
49 def support():
50     print("[SUPPORT] use the two provided functions as black boxes; fill this table only:")
51     print(f"    out_size(96,3,1,1)=? out_size(96,3,2,1)=? rf of four 3x3 layers=?")
52     print("    answers:", out_size(96,3,1,1), out_size(96,3,2,1), rf_trace([(3,1)]*4))
53
54 def extension():
55     def rf_dilated(layers):
56         rf, jump = 1, 1
57         for k, s, d in layers: rf += d*(k-1)*jump; jump *= s
58         return rf
59     print("[EXTENSION] add dilation to rf(): three 3x3 layers with dilation 1,2,4 ->",
60           rf_dilated([(3,1,1),(3,1,2),(3,1,4)]), "(plain three 3x3 gave 7)")
61     print("    Same 27 weights. Where is the catch? (Hint: gridding.)")
62
63 if __name__ == "__main__":
64     import sys
65     if len(sys.argv) > 1 and sys.argv[1] == "support": support()
66     elif len(sys.argv) > 1 and sys.argv[1] == "extension": extension()
```

Output:

```
$ python tut9.py
[1] k7 s2 p0 on 224: scipy gives 109, out_size() gives 109
    k7 s2 p3 -> 112
    k3 s2 p1 -> 56
    k3 s1 p1 -> 56
    k3 s2 p1 -> 28
[3] RF of two 3x3: 5
[3] RF of three 3x3: 7
[3] RF of with strides: 21
[4] plain 3x3 layers for RF>=100: 50
    alternating stride-2:      10 layers
```